

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 20

Sorting Networks

Sorting

- So far:
 - **Selection sort** and **Heap Sort**: $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable. Heap Sort is a fall back for `std::sort`
 - **Insertion sort** and **Tree Sort**: $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable. Insertion sort used by `std::sort` and `std::stable_sort` to handle the base-case.
 - **Merge Sort**: $O(n \log n)$. Not in-place. Stable. Main component of `std::stable_sort`
 - **Quicksort**: $O(n \log n)$ on average, or using certain pivot selection schemes (not). Main component of `std::sort`

Bubble Sort

- Another simple $O(n^2)$ -time in-place sorting algorithm (cf. selection sort and insertion sort)
 - Moves largest items to their final location, one by one (like selection sort)
 - Relative order of two items flipped only if required by sorting order, making it stable (unlike selection sort)

```
void sortpair(int& a, int& b) {  
    if (a>b) std::swap(a,b)  
}  
  
void bubblesort (std::vector<int>& A) {  
    for(int k=A.size(); k > 1; --k)  
        for(int i=0; i<k-1; ++i)  
            sortpair(A[i],A[i+1]);  
}
```

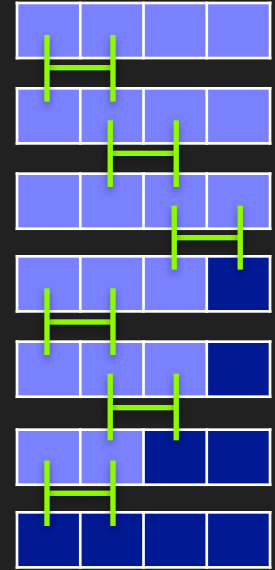


Exercise:

1. Exit early, when already sorted.
2. Replace calls to sortpair with shifting to save on number of assignments.

Bubble Sort

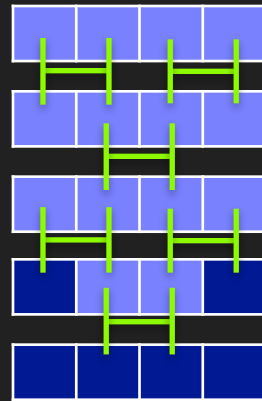
- The order of comparisons/moves doesn't depend on the data!
- Can build a "circuit" using "sortpair gates" (comparators) for sorting
 - Such a circuit is called a sorting network
- Complexity measures: Number of comparators (hardware cost), number of levels (time taken)



```
void sortpair(int& a, int& b) {  
    if (a>b) std::swap(a,b)  
}  
  
void bubblesort (std::vector<int>& A) {  
    for(int k=A.size(); k > 0; --k)  
        for(int i=0; i<k-1; ++i)  
            sortpair(A[i],A[i+1]);  
}
```

Sorting Networks

- Can improve on the sorting network from bubble sort
 - Instead of $O(n^2)$ time, $O(n)$ levels enough
 - With $O(n)$ comparators at each level
 - Odd-Even Transposition Network
- How do we know this works?
 - Two-step argument:
 - **0-1 Principle:** Enough to analyse inputs in $\{0,1\}^n$
 - No error after n steps in a $\{0,1\}^n$ input



Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

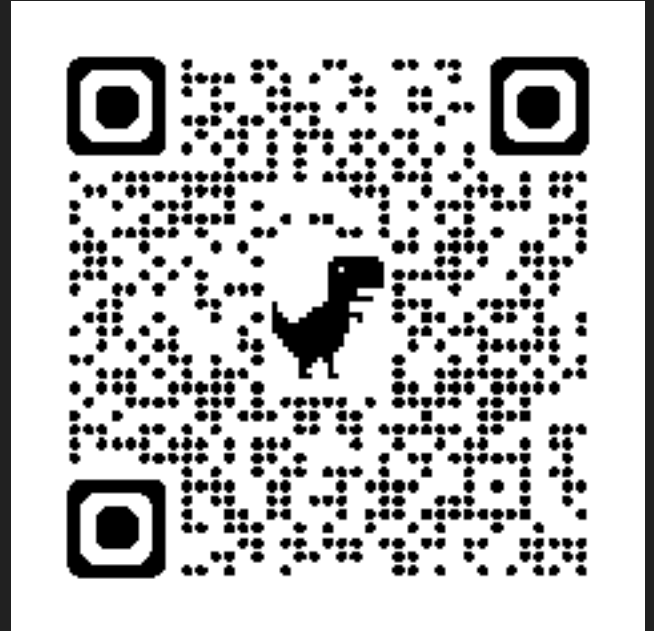
Read the question, choose the answer and submit.

The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

**If you are not in the classroom, a report will
be sent to DADAC!**



0-1 Principle

- If a sorting network correctly sorts all 0-1 sequences, then it correctly sorts all inputs
- Contrapositive: If a sorting network is wrong on some input, then it must be wrong on some 0-1 sequence
- Proof: Suppose on input $(x_1, \dots, x_n)$ it is wrong. Then some inputs a and b such that $a < b$ that ended up in the wrong order (b before a)
 - Consider input z such that $z_i = T(x_i)$ where $T(x) = \begin{cases} 0 & \text{if } x \leq a \\ 1 & \text{otherwise.} \end{cases}$
 - Inductively, at any level, results while sorting x and z , are related by $z^{(t)} = T(x^{(t)})$: because $\min(T(u), T(v)) = T(\min(u, v))$ and $\max(T(u), T(v)) = T(\max(u, v))$.
 - At the final level, $T(b)=1$ before $T(a)=0$

n Steps for Odd-Even Transposition Network

- Two-step argument to prove a sorting network is correct:
 - **0-1 Principle:** For any sorting network, enough to analyse inputs in $\{0,1\}^n$
 - The given network correctly sorts every input in $\{0,1\}^n$
- Claim: The odd-even transposition network on n inputs, with n levels, correctly sorts every input in $\{0,1\}^n$
 - The rightmost 1 may or may not move in the first step (depending on if it is compared to the left or to the right); after that it will *start flowing*: i.e., move to the right at every step until it reaches its final position, $n-1$
 - Inductively: The k^{th} -from-right 1 (whose final position is $n-k$) will start flowing after at most k steps, because it will not be blocked by a 1 to its right that is already flowing (but could stall once before starting to chase it)
 - All 1s reach final positions in at most n steps: $\leq k$ steps before flowing, and $\leq n-k$ steps in the flow

Sorting Networks

- **Odd-even transposition network** takes $O(n)$ time using $O(n)$ processors in parallel
 - As a circuit, needs $O(n^2)$ comparators
 - Can do much better! (cf. $\Omega(n \log n)$ lower bound)
- **Batcher's Bitonic sorting & Batcher's Odd-Even Mergesort**
 - **$O(\log^2 n)$ time**, using $O(n)$ processors in parallel (total $O(n \log^2 n)$ comparators).
 - Practical (small constants hidden by O). Used in GPUs and circuits, for moderately large n .
- **In principle, $O(\log n)$ time using $O(n)$ processors enough!**
 - But the hidden constants possible for the best known algorithms are in 1000s (where as $\log_2 n$ is *well under* 100 for all practical n)